

Übungsblatt 6: Hardware-Beschleunigung (DMA)

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen Vorlesung 6: Direct Memory Access (Interrupt Storm, Bus-Mastering/Cycle Stealing, Cache-Kohärenz, Ringpuffer/Double Buffering) sowie den Vergleich Polling vs. DMA-UART in C.

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 14 P, Aufgabe 2: 10 P (Gesamt: 24 P)

Teil 1: Theorie — Direct Memory Access (DMA)

Aufgabe 1.1: Der CPU-Flaschenhals

Große Datenmengen (z. B. ein 1 MB-Bild) über schnelle SPI an ein Display zu senden ist per Software-Polling oder Interrupt problematisch.

1. Warum führt interrupt-basierter Versand bei sehr schnellen Peripheriemodulen (SPI im Megahertz-Bereich) zu einem *Interrupt Storm*, der die CPU auslastet?
2. Inwiefern entlastet ein DMA-Controller das System im Vergleich zur klassischen Befehlskette $1w/sw$ durch die ALU?

Musterlösung Aufgabe 1.1

1. Ein schnelles SPI-Modul meldet im Sub-Mikrosekunden-Takt (z. B. alle $0,8\mu s$), dass das Senderegister leer ist. Interrupt-basiert wird die CPU millionenfach pro Sekunde unterbrochen. Kontextwechsel (Pipeline flush, Register sichern/wiederherstellen) kostet Dutzende bis Hunderte Takte *pro Byte* — die CPU ertrinkt in ISR-Overhead (*Interrupt Storm*) und kommt kaum noch zum Rechnen.
2. Bei $1w/sw$ wandert jedes Byte physisch durch Register-File/ALU der CPU. Der DMAC transportiert autonom SRAM $\rightarrow$ Peripherie und führt den Datenstrom an der CPU vorbei (*Bypass*). Die CPU-Last für den Transfer sinkt von nahe 100 % Richtung nahe 0 %.

Aufgabe 1.2: Bus-Mastering und Arbitration

Ein DMA-Controller agiert als eigenständiger Master auf dem Systembus.

1. Was ist ein *Bus-Master* im Gegensatz zu einem passiven Peripheriegerät (Slave)?
2. Wie löst der Bus-Arbiter Konflikte, wenn CPU und DMA im selben Taktzyklus auf denselben SRAM zugreifen? Was ist *Cycle Stealing*?

Musterlösung Aufgabe 1.2

1. Ein *Bus-Master* darf selbst Transaktionen starten (Adressen/Steuerleitungen treiben) — typisch CPU und DMAC. Ein Slave (z. B. UART-Datenregister) reagiert nur, wenn ein Master ihn anspricht; er fordert keine Buszugriffe von sich aus an.
2. Der Arbiter (Bus-Matrix) schlichtet nach Priorität. Oft hat der DMA Vorrang (harte Echtzeit, sonst Datenverlust). Der verlierende Master (häufig die CPU) wird kurz angehalten (*Stall*). *Cycle Stealing*: der DMA „klaut“ einzelne Buszyklen; dank Caches ist der Rechenverlust oft kaum messbar.

Aufgabe 1.3: Cache-Kohärenz bei DMA-Transfers

Bei Write-Back-Data-Caches kann die Kombination Cache + DMA zu Fehlern führen.

1. Erklären Sie *Stale Data* beim Senden (CPU → DMA) und beim Empfangen (DMA → CPU).
2. Nennen Sie zwei gängige Software- bzw. Hardware-Methoden zur Cache-Kohärenz bei DMA.

Musterlösung Aufgabe 1.3

1. **Senden:** CPU schreibt in den Write-Back-Cache; SRAM bleibt veraltet → DMA liest alte Bytes (*Stale Data*).
Empfangen: DMA schreibt frische Daten in den SRAM; CPU trifft noch den alten Cache-Eintrag und sieht Müll.
2. **Software-Cache-Wartung:** vor Senden *Clean/Flush* der Puffer-Lines; vor Lesen empfangener Daten *Invalidate*.
Uncached-Regionen: DMA-Puffer per Linker/MPU/PMP in uncached RAM legen — CPU greift immer direkt auf RAM zu.

Aufgabe 1.4: Fortgeschrittene Puffer-Konzepte

Für endlose Streams (z. B. Audio mit 44100 Samples/s) reichen lineare Einmal-Puffer nicht.

1. Wie funktioniert der *Circular Mode* (Ringpuffer)? Welche Rolle hat *Auto-Wrap*?
2. Erklären Sie *Double Buffering* (Ping-Pong) zusammen mit dem *Half-Transfer-Interrupt* (HT).

Musterlösung Aufgabe 1.4

1. Im Circular Mode läuft ein SRAM-Puffer als Ring. Erreicht COUNT den Wert 0, stoppt der Kanal nicht, sondern setzt den Zeiger per Hardware auf den Pufferanfang zurück (*Auto-Wrap*) und beginnt erneut.
2. Der Ring wird logisch in zwei Hälften geteilt. Während der DMA eine Hälfte ausspielt (z. B. an einen DAC), füllt die CPU die andere. HT feuert in der Puffermitte, TC am Ende (bei Circular oft am Wrap) — Ping-Pong für glitchfreien Dauerstrom.

Aufgabe 1.5: DMA-Durchsatz und CPU-Entlastung

Ein SPI-Controller arbeitet mit 10 MHz Bitrate und überträgt 1 MiB Bilddaten an ein Display. Vereinfachung: 1 Bit pro Takt, kein Protokoll-Overhead.

1. Wie lange dauert der Transfer (in Sekunden)?
2. Bei Polling blockiert die CPU während des gesamten Transfers (*Busy-Waiting*). Wie lange ist die CPU damit blockiert?
3. Mit DMA übernimmt der DMAC den Transfer. Was gilt näherungsweise für die CPU-Last während des Transfers — und wofür braucht die CPU dennoch Zyklen?

Musterlösung Aufgabe 1.5

1. Bitrate 10 Mbit/s, Datenmenge 1 MiB = 8 Mbit:

$$t = \frac{8 \times 10^6 \text{ bit}}{10 \times 10^6 \text{ bit/s}} = 0,8 \text{ s}$$

(Mit 1 MB = 10^6 Byte analog: $t = 0,8 \text{ s}$.)

2. Polling: die CPU ist die gesamten 0,8 s blockiert (nahe 100 % Last für den Transfer).
3. Mit DMA: CPU-Last für den Datentransfer selbst ≈ 0 . Die CPU braucht nur kurze Setup-Zyklen (Register konfigurieren, ggf. Cache Clean, Kanal starten) und optional eine Transfer-Complete-ISR am Ende.

Teil 2: Praxis — DMA-Konfiguration und Polling-Vergleich

Szenario: Ein String-Puffer soll über UART ausgegeben werden — zuerst blockierend per Polling, dann per DMA.

Gegeben (didaktisch/fiktiv):

- UART-Basis: 0x40010000 — Status +0x00, TX-Daten +0x04, Control +0x08
- DMA-Kanal 1: 0x40020000 — Felder SRC_ADDR, DST_ADDR, COUNT, CONTROL

Aufgabe 2.1: Blockierende Polling-Schleife

Die Funktion blockiert, bis das UART-TX-Register jeweils bereit ist (UART_TX_EMPTY).

```
#include <stdint.h>

#define UART_BASE      0x40010000u
#define UART_STATUS_REG (*(volatile uint32_t *) (UART_BASE + 0x00u))
#define UART_TX_REG    (*(volatile uint32_t *) (UART_BASE + 0x04u))
#define UART_TX_EMPTY  (1u << 0) /* 1 = TX-Register leer */

void send_uart_polling(const char *text, uint32_t len) {
    for (uint32_t i = 0; i < len; i++) {
        /* 1. Warten, solange TX noch voll ist */
        /* TODO: while (...) { } */

        /* 2. Zeichen schreiben */
        /* TODO: UART_TX_REG = ...; */
    }
}
```

Musterlösung Aufgabe 2.1

```
#include <stdint.h>

#define UART_BASE      0x40010000u
#define UART_STATUS_REG (*(volatile uint32_t *) (UART_BASE + 0x00u))
#define UART_TX_REG    (*(volatile uint32_t *) (UART_BASE + 0x04u))
#define UART_TX_EMPTY  (1u << 0)

void send_uart_polling(const char *text, uint32_t len) {
    for (uint32_t i = 0; i < len; i++) {
        while ((UART_STATUS_REG & UART_TX_EMPTY) == 0u) {
            /* Busy-Waiting */
        }
        UART_TX_REG = (uint32_t)text[i];
    }
}
```

Aufgabe 2.2: UART für DMA konfigurieren

Das UART muss Hardware-Requests (DREQ) ausgeben, sobald das Senderegister leer ist.

```
#define UART_CR3 (*(volatile uint32_t *) (UART_BASE + 0x08u))
#define UART_DMAR (1u << 7) /* DMA Request Enable */

void uart_enable_dma_req(void) {
    /* TODO: UART_CR3 |= ...; */
}
```

Musterlösung Aufgabe 2.2

```
#define UART_CR3 (*(volatile uint32_t *) (UART_BASE + 0x08u))
#define UART_DMAR (1u << 7)

void uart_enable_dma_req(void) {
    UART_CR3 |= UART_DMAR;
}
```

Aufgabe 2.3: DMA-Kanal initialisieren und starten

Quelle: String-Array; Ziel: UART-TX. Quelladresse inkrementieren, Zieladresse fest (Memory-to-Peripheral).

```
#include <stdint.h>

#define UART_BASE    0x40010000u
#define UART_TX_REG (*(volatile uint32_t *) (UART_BASE + 0x04u))

typedef struct {
    volatile uint32_t SRC_ADDR;
    volatile uint32_t DST_ADDR;
    volatile uint32_t COUNT;
    volatile uint32_t CONTROL;
} DMA_Channel_TypeDef;

#define DMA_CH1      ((DMA_Channel_TypeDef *)0x40020000u)
#define DMA_MINC_EN   (1u << 0) /* Memory Increment */
#define DMA_DIR_M2P   (1u << 1) /* Memory-to-Peripheral */
#define DMA_TCIE_EN   (1u << 2) /* Transfer Complete IRQ */
#define DMA_ENABLE_BIT (1u << 31) /* Channel Enable */

void start_uart_dma_transfer(const char *text, uint32_t len) {
    /* 1. Quell- und Zieladresse */
    /* TODO: DMA_CH1->SRC_ADDR = ...; */
    /* TODO: DMA_CH1->DST_ADDR = ...; */

    /* 2. Transferlänge */
    /* TODO: DMA_CH1->COUNT = ...; */

    /* 3. Steuerung: MINC, M2P, TCIE */
    /* TODO: DMA_CH1->CONTROL = ...; */

    /* 4. Enable-Bit setzen */
    /* TODO: DMA_CH1->CONTROL |= ...; */
}
```

Musterlösung Aufgabe 2.3

```
#include <stdint.h>

#define UART_BASE    0x40010000u
#define UART_TX_REG (*(volatile uint32_t *) (UART_BASE + 0x04u))

typedef struct {
    volatile uint32_t SRC_ADDR;
    volatile uint32_t DST_ADDR;
    volatile uint32_t COUNT;
    volatile uint32_t CONTROL;
} DMA_Channel_TypeDef;
```

```

#define DMA_CH1          ((DMA_Channel_TypeDef *)0x40020000u)
#define DMA_MINC_EN      (1u << 0)
#define DMA_DIR_M2P      (1u << 1)
#define DMA_TCIE_EN      (1u << 2)
#define DMA_ENABLE_BIT   (1u << 31)

void start_uart_dma_transfer(const char *text, uint32_t len) {
    DMA_CH1->SRC_ADDR = (uint32_t)text;
    DMA_CH1->DST_ADDR = (uint32_t)&UART_TX_REG;

    DMA_CH1->COUNT = len;

    DMA_CH1->CONTROL = DMA_MINC_EN | DMA_DIR_M2P | DMA_TCIE_EN;
    DMA_CH1->CONTROL |= DMA_ENABLE_BIT;
}

```

(Hinweis: &UART_TX_REG ist didaktisch gemeint als Adresse des TX-Datenregisters; in realem Code oft ein konstanter MMIO-Zeiger. Vor dem Start typisch noch `uart_enable_dma_req()` aufrufen.)